

What we used — and from where

The complete inventory: the analytical methods, the technology & libraries (with where each is used in the code and where it comes from), and the external data sources with their exact origins.

Pure Python · no web framework · no database · no machine learning in the live system

01 Methods — the techniques we used

The system is built on plain statistics and exhaustive search — nothing learned or fitted.

METHOD	WHAT IT DOES	USED IN
Event-study window	Measures price change in a window around each corporate event.	event_behaviour.py
Exhaustive grid search	Tests all 64 buy-before / sell-after timing rules (N,M = 1...8) — no optimiser, just try them all.	event_behaviour.py
Descriptive statistics	Win rate (share of events that finished positive), average, median, best/worst case.	event_behaviour.py · stock_server.py
Trading-day indexing	Counts in market days, skipping weekends & holidays.	event_behaviour.py
Market adjustment	Subtracts the NIFTY index move to isolate stock-specific reaction.	nifty_backtest.py · event_study.py
Train / test + placebo	Splits history and tests random dates to check an edge isn't just curve-fitting.	nifty_backtest.py
Fingerprinted disk cache	Persists rankings/screener keyed by a data fingerprint → instant loads, auto-refresh on change.	stock_server.py

02 Technology & libraries — what, from where, used where

STDLIB ships with Python PyPI installed via pip EXTERNAL separate program OWN written in this project

TOOL	PURPOSE	FROM WHERE	USED IN
Python 3	The whole backend language.	EXTERNAL python.org	all 32 scripts
pandas	Every table, CSV read/write, grouping, dates.	PyPI pip install pandas	~all scripts
numpy	Numeric arrays & math.	PyPI numpy	event/optimal/earnings scripts
matplotlib	A few static plot images.	PyPI matplotlib	straddle_backtest.py
openpyxl	Builds the downloadable Excel workbooks.	PyPI openpyxl	stock_server.py
scikit-learn	ML <i>experiment only</i> — not in the live system.	PyPI scikit-learn	ml_model.py (standalone)
urllib · ssl · json	Fetch from Yahoo/NSE/Screener; parse responses.	STDLIB	fetchers + server
http.server	The web server behind the dashboard — no framework.	STDLIB	stock_server.py
HTML · CSS · JS · SVG	The dashboard UI & charts, hand-written (no React).	OWN	stock_server.py (PAGE)
Headless Chrome	Renders the HTML docs to PDF.	EXTERNAL Google Chrome	documentation / report PDFs
Git + GitHub Pages	Version control & the always-on static host.	EXTERNAL github.com	docs/nifty/

No web framework, no database, no ML in the product. The dashboard is Python's built-in `http.server` serving hand-written HTML/JS; data lives in plain CSV files plus a JSON disk-cache.

03 Data sources — where the data comes from

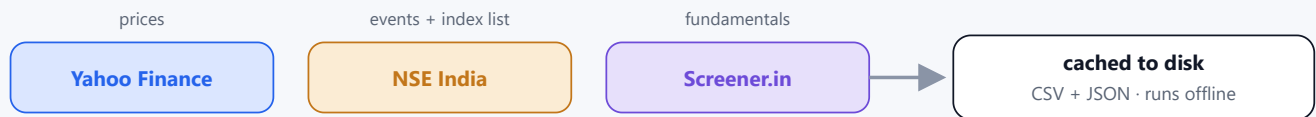


Figure 1. Three external sources, all cached locally so the analysis runs without internet.

DATA	SOURCE	FROM WHERE (ENDPOINT / FILE)	CACHED AS
Daily prices	Yahoo Finance	<code>query1.finance.yahoo.com/v8/finance/chart/<SYM>.NS</code>	<code>processed/price_cache/</code>
NIFTY index	Yahoo Finance	<code>.../chart/^NSEI</code>	<code>price_cache/_NSEI.csv</code>
Event feeds	NSE India	<code>nseindia.com/api/corporates-*</code> + the NSE export (new data/*.xlsx)	<code><SYM>/*.csv</code>
Fundamentals	Screener.in	<code>screener.in/company/<SYM>/</code> + <code>/api/company/<id>/schedules/</code>	<code>pnl/ balance_sheet/ cash_flow/ ratios/ quarterly/</code>
Nifty 50 list	NSE India	<code>equity-stockIndices?index=NIFTY 50</code> · Company Directory export	<code>download_feeds.py</code>

Prices are adjusted close (corrected for splits/bonuses) and everything is snapshotted to disk — so once downloaded the whole system, dashboard included, works fully offline.

04 What we built ourselves

OUR SCRIPT	WHAT IT DOES
<code>event_behaviour.py</code>	The engine — the 64-rule grid of returns & win rates per event.
<code>stock_server.py</code>	The dashboard app — payloads, feeds, rankings, screener, compare, caching.
<code>nifty_dash.py</code>	The Nifty-50 front end (port 8090) — rebrands + warms caches.
<code>download_feeds.py</code>	The Nifty 50 list + NSE feed downloader.
<code>import_new_data.py</code>	Splits the NSE Excel export into per-stock CSVs.
<code>merge_event_history.py</code>	Unions the export with the archive (recovers full history).
<code>enrich_board_meetings.py</code>	Backfills board meetings from the announcements archive.
<code>build_static_nifty.py</code>	Bakes the dashboard into a static GitHub Pages site.
<code>fund_loader.py</code>	Loads the Screener fundamentals folders.